

# Práctica del Laboratorio 1

## Arquitectura del Computador

29 de Julio de 2026

### 1. ¿Cómo se implementa la recursividad en MIPS32?

La recursividad en MIPS32 se implementa utilizando la pila (stack) para preservar el estado de cada llamada recursiva. El registro \$sp (stack pointer) es fundamental en este proceso.

#### ¿Qué papel cumple la pila (\$sp)?

El stack pointer (\$sp) cumple estas funciones clave:

- **Almacenamiento de direcciones de retorno:** Guarda \$ra para saber dónde retornar después de cada llamada.
- **Preservación de registros:** Guarda valores de registros que podrían modificarse en llamadas posteriores.
- **Almacenamiento de parámetros y variables locales:** Para cada instancia de la llamada recursiva.

### 2. ¿Qué riesgos de desbordamiento existen?

#### 1. Desbordamiento Aritmético (Arithmetic Overflow)

Ocurre cuando el resultado de una operación supera los 32 bits, provocando errores en el bit de signo o pérdida de datos.

##### Cómo mitigarlo (Ejemplo en MIPS):

En lugar de utilizar `add $t0, $t1, $t2` (que dispara excepción), emplear `addu $t0, $t1, $t2`. Esta instrucción suma los bits sin verificar el acarreo hacia el bit de signo, evitando interrupciones inesperadas. Para validar manualmente, se puede usar `slt $t3, $t1, $zero` y comparar con el resultado, asegurando que el programa maneje el error mediante un salto lógico antes de guardar valores corruptos.

## 2. Desbordamiento de Estructura (Stack Overflow)

Se origina cuando la recursión excesiva agota la memoria reservada para el puntero de pila (\$sp), invadiendo segmentos de memoria ajenos.

**Cómo mitigarlo (Ejemplo en MIPS):**

- **Caso base:** Usar `beq $a0, $zero`, finparidad para detener la recursión inmediatamente.
- **Validación de rango:** Emplear `bltz $a0, fin` al inicio para evitar que números negativos provoquen una recursión infinita.
- **Contador de seguridad:** Se puede utilizar `slti $t0, $s1, 100` (donde \$s1 es un contador de profundidad) antes de ejecutar el `jal paridad`. Si el contador llega a 100, el programa salta a una rutina de error (`j error`) antes de que el \$sp corrompa la memoria.

## 3. Desbordamiento de Tipo (Type Overflow / Truncation)

Ocurre al intentar comprimir datos de 64 bits (procedentes de `mult`) en registros de 32 bits, o por el uso de direcciones mal alineadas.

**Cómo mitigarlo en MIPS 32:**

- Extraer los bits superiores con `mfhi` y los bits inferiores con `mflo` después de una multiplicación.
- Verificar que el valor en `mfhi` sea igual a la extensión de signo del registro LO.
- Guardar ambos registros (HI y LO) por separado en memoria si se necesita preservar el resultado completo.
- Asegurar que los offsets sean siempre múltiplos de 4 al usar `lw` o `sw` para evitar errores de alineación.

## 3. ¿Qué diferencias encontraste entre una implementación iterativa y una recursiva en cuanto al uso de memoria y registros?

### 1. Uso de Memoria

**Iterativa:** Es mucho más eficiente en cuanto a memoria. El código iterativo no necesita la pila. Todo el trabajo se realiza usando unos pocos registros (como \$t0, \$t1) que se sobrescriben en cada vuelta del bucle. No importa si el número es pequeño o grande; el consumo de memoria es siempre constante ( $O(1)$ ).

**Recursiva:** Es costosa. Cada vez que la función se llama a sí misma, el programa debe reservar espacio en la pila (\$sp) para guardar la dirección de retorno (\$ra) y los argumentos. Si el número a evaluar es grande, la pila crece linealmente ( $O(n)$ ), lo que consume RAM rápidamente.

## 2. Uso de Registros

**Iterativa:** Los registros son reutilizados. Una vez que el bucle pasa a la siguiente iteración, el valor anterior ya no es necesario, por lo que se reemplaza. Es un proceso "destrutivo" pero muy rápido.

**Recursiva:** Los registros deben ser "preservados". Antes de llamar a la siguiente instancia de la función, el código debe copiar el valor actual de los registros a la memoria (pila) para "congelar" el estado. Al volver de la recursión, debe restaurarlos (`lw`). El procesador gasta más tiempo moviendo datos entre registros y memoria que haciendo la operación matemática en sí.

## 4. ¿Qué diferencias encontraste entre los ejemplos académicos del libro y un ejercicio completo y operativo en MIPS32?

Al comparar los ejemplos académicos con nuestra implementación real, identificamos tres diferencias clave:

1. **El entorno obligatorio:** A diferencia de los fragmentos de código del libro, nuestro programa requirió directivas obligatorias (`.data`, `.text`, `.globl main`) para que el simulador pudiera asignar memoria y ejecutar la lógica.
2. **La optimización de registros:** Mientras el libro prioriza el uso de registros \$s (que requieren respaldo en pila), nosotros utilizamos registros \$t en la versión iterativa. Esto eliminó instrucciones `sw/lw` innecesarias, haciendo el código más ágil y eficiente.
3. **Interacción con el usuario:** Los ejemplos del libro se limitan a cálculos matemáticos puros. En la práctica, tuvimos que integrar `syscalls` (servicios del sistema) para leer entradas y mostrar resultados, gestionando registros adicionales como \$v0 y \$a0.

## 5. Elaborar un tutorial de la ejecución paso a paso en MARS.

Este procedimiento aplica para el análisis de nuestros programas de paridad (iterativa y recursiva):

1. **Ensamblado:** Abra el archivo `.asm` en MARS, acceda al menú Run y seleccione Assemble (F3). Esto prepara el código para su ejecución.

2. **Preparación del entorno:** Asegúrese de tener visibles las pestañas Registers (para monitorear cambios en \$t, \$a y \$v) y Data Segment (esencial para observar el comportamiento de la pila \$sp en el ejercicio recursivo).
3. **Depuración (Single Step):** Utilice el botón Single Step (F7) para ejecutar una instrucción a la vez.
  - En el iterativo: Verifique cómo el registro \$t3 almacena el residuo tras la operación `div`.
  - En el recursivo: Observe cómo el valor de \$sp decrece en la ventana Data Segment con cada llamada (`jal`), indicando la reserva de memoria, y cómo se recuperan los datos al ejecutar `lw`.
4. **Validación de resultados:** Una vez alcanzada la instrucción `syscall`, observe la consola Run I/O para confirmar que el resultado mostrado coincide con el valor calculado en los registros.
5. **Reset:** Utilice el botón Reset para limpiar el estado y realizar nuevas pruebas con distintos números, garantizando que el programa sea consistente en cualquier escenario.

## 6. Justificar la elección del enfoque (iterativo o recursivo) según eficiencia y claridad en MIPS.

Tras implementar ambos métodos, elegimos el enfoque iterativo como el estándar óptimo por dos razones principales:

1. **Eficiencia técnica:** La versión iterativa opera exclusivamente mediante registros, eliminando el alto costo de memoria (escritura/lectura en RAM) que exige la gestión de la pila en la recursión. Esto reduce significativamente los ciclos de reloj.
2. **Claridad y seguridad:** Aunque la recursión es matemáticamente elegante, su gestión manual en ensamblador es propensa a errores. El enfoque iterativo es más directo, fácil de seguir paso a paso y elimina por completo el riesgo de un Stack Overflow, garantizando un funcionamiento robusto sin importar el tamaño del número ingresado.

## 7. Análisis y Discusión de los Resultados

### Resultados cuantitativos

Para un valor  $n$  cualquiera dentro del rango de 32 bits, observamos la siguiente diferencia en la ejecución:

Table 1: Resultados cuantitativos (Paridad)

| n  | Ciclos (Iterativo) | Ciclos (Recursivo) |
|----|--------------------|--------------------|
| 10 | ~20                | ~150               |
| 20 | ~20                | ~300               |
| 30 | ~20                | ~450               |

Table 2: Comparativa técnica de enfoques

| Métrica                | Paridad Iterativa | Paridad Recursiva |
|------------------------|-------------------|-------------------|
| Complejidad de Tiempo  | $O(1)$            | $O(n)$            |
| Consumo de Ciclos      | Constante         | Lineal            |
| Uso de Memoria (Stack) | Nulo              | $O(n)$            |

## Hallazgos principales

**Límites arquitecturales:** Ambos métodos están limitados por la arquitectura de 32 bits de MIPS. Si el valor ingresado excede el rango máximo del registro ( $2^{31} - 1$ ), se producirá un overflow aritmético en ambos casos. Además, en la versión recursiva, se presenta un límite adicional de Stack Overflow cuando  $n$  es lo suficientemente grande como para agotar el espacio reservado para la pila en el simulador.

### Optimizaciones efectivas:

- En el iterativo: Implementamos el uso directo de registros temporales (\$t), eliminando el acceso a memoria RAM y acelerando el cálculo.
- En el recursivo: Se minimizó el uso de registros guardados (\$s), optimizando el respaldo en la pila solo a lo estrictamente necesario (\$ra y \$a0).

## Conclusiones

La implementación iterativa es superior a la recursiva por los siguientes motivos:

1. **Eficiencia computacional:** Al ser un proceso de tiempo constante ( $O(1)$ ), no depende del tamaño de  $n$ , ejecutándose mucho más rápido que la recursiva, que debe realizar una llamada por cada unidad de  $n$ .
2. **Optimización de recursos:** El enfoque iterativo opera exclusivamente en registros. Esto evita el consumo de memoria en la pila, lo cual es crítico para la estabilidad del sistema.
3. **Estabilidad:** La versión iterativa es más robusta frente a la profundidad de llamadas. Mientras que la recursión está sujeta a fallos por desbordamiento de memoria de pila incluso antes de alcanzar los límites aritméticos del procesador, el método iterativo aprovecha al máximo el espacio de memoria disponible en la arquitectura.